Programación declarativa

• Haskell •

Doble grado en Ingeniería Informática y Matemáticas
Universidad Complutense de Madrid
Óscar Martín — Curso 2026/27

26. Archivos

- `IO` también tiene funciones para el manejo de archivos.
- `type FilePath = String`
- `writeFile :: FilePath -> String -> IO ()`
  - Crea el archivo o lo sobrescribe.
- `appendFile :: FilePath -> String -> IO ()`
  - Como `writeFile` pero añade en vez de sobrescribir.
- `readFile :: FilePath -> IO String`
  - El resultado de la acción es (*conceptualmente*) el contenido *completo* del archivo.

```haskell
transformaArch :: (String -> String) -> IO ()
transformaArch f = do
    putStrLn "Nombre del archivo:"
    nombre <- getLine
    contenido <- readFile nombre
    writeFile (nombre ++ ".nuevo") (f contenido)
```

  - Por ejemplo: `transformaArch reverse`.
- Pero `readFile` funciona de forma perezosa: solo lee lo que va necesitando.

```
ghci> :set +s
ghci> writeFile "grande.txt" (unlines (map show [1..10000000]))
(4.74 secs, 8,713,932,416 bytes)
ghci> readFile "grande.txt" >>= putStrLn . head . lines
1
(0.00 secs, 170,360 bytes)
ghci> readFile "grande.txt" >>= print . length . lines
10000000
(1.31 secs, 13,437,849,936 bytes)
```

- `getContents :: IO String` hace lo mismo para la entrada estándar.

Ejercicios

1. Define `cuentaArchivo :: FilePath -> IO ()`, que escribe cuántas líneas, palabras y caracteres tiene un archivo: `cuentaArchivo "poema.txt"` escribe `4 líneas, 21 palabras, 112 caracteres.`

2. Define `ponAlFinal :: FilePath -> IO ()`, que lee líneas del teclado hasta que lee una vacía y las añade al final de un archivo.

3. ¿Qué pasa si pedimos a `transformaArch` que escriba en el mismo archivo del que está leyendo?